

CENTRO UNIVERSITÁRIO INTERNACIONAL UNINTER
TECNOLOGIA EM ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

ÍCARO BOMFIM TELES

PROJETO MULTIDISCIPLINAR – SGHSS

VidaPlus: Sistema de Gestão Hospitalar e de Serviços de Saúde

Ênfase: Desenvolvimento Front-end

Professor Orientador: Winston Sen Lun Fung Feira de Santana/Bahia

Feira de Santana/Bahia

2026/1

1. Introdução

Contexto

A **VidaPlus** é uma instituição que administra hospitais, clínicas, laboratórios e equipes de home care. Diante da necessidade de modernizar a gestão de seus serviços de saúde, foi proposto o desenvolvimento do **VidaPlus** — um protótipo funcional de **Sistema de Gestão Hospitalar e de Serviços de Saúde (SGHSS)**.

O projeto foi desenvolvido como trabalho acadêmico do curso de Tecnologia em Análise e Desenvolvimento de Sistemas do Centro Universitário Internacional **Uninter**, com ênfase em **Desenvolvimento Front-end**.

Objetivo

Construir uma aplicação web funcional que demonstre os fluxos essenciais de gestão hospitalar:

- **Cadastro e gerenciamento de pacientes** com validação de CPF
- **Agendamento de consultas** com controle de status
- **Prontuário eletrônico** com histórico clínico
- **Receitas digitais** com prescrições médicas
- **Teleconsulta real** via videoconferência (Jitsi Meet)
- **Gestão administrativa** com relatórios e métricas
- **Controle de acesso por perfil** (RBAC)
- **Conformidade LGPD** com avisos e consentimento

Usuários do sistema

O VidaPlus atende três perfis de usuário com permissões distintas:

Perfil	Responsabilidades
Administrador	Cadastro de pacientes e profissionais, relatórios, gestão completa
Paciente	Agendamento de consultas, visualização de prontuário, teleconsulta
Profissional de saúde	Registro em prontuários, emissão de receitas, teleconsulta, agenda

Escopo e limitações

Por ser um projeto acadêmico com **ênfase em Desenvolvimento Front-end**, o backend é mínimo (Server Actions + SQLite) e serve como suporte para demonstrar os fluxos de interface. Algumas funcionalidades mencionadas no roteiro do projeto foram **priorizadas ou diferidas** conforme a orientação de que não é obrigatório implementar tudo, desde que as escolhas sejam justificadas:

Funcionalidade	Decisão	Justificativa
Cadastro e atendimento	✓ Incluída	Requisito essencial do fluxo hospitalar
Consultas e agendamentos	✓ Incluída	Requisito essencial
Prontuário eletrônico	✓ Incluída	Requisito essencial
Teleconsulta (videoconferência)	✓ Incluída	Diferencial técnico relevante (Jitsi Meet real)
Receitas digitais	✓ Incluída	Requisito essencial para o fluxo clínico
Gestão de profissionais	✓ Incluída	CRUD completo para demonstrar gestão administrativa
Notificações (48h)	✓ Incluída	Demonstra integração com dados temporais
RBAC e LGPD	✓ Incluída	Requisitos transversais obrigatórios
Gestão de leitos	✗ Diferida	Escopo extenso para MVP; requer modelagem hospitalar específica além do front-end
Relatórios financeiros	✗ Diferida	Requer integração com sistema de faturamento inexistente no MVP
Controle de suprimentos	✗ Diferida	Domínio de logística hospitalar fora do foco front-end
Criptografia de dados clínicos	⚠ Parcial	Senhas com bcrypt; dados clínicos em texto plano (aceitável para SQLite local em ambiente acadêmico)
Registros de auditoria (logs)	✗ Diferida	Planejado como evolução futura; createdAt já rastreia criação de registros

O foco do MVP foi entregar **fluxos completos e visualmente polidos** para os três perfis de usuário, com **testes automatizados** e **responsividade mobile**, demonstrando competência técnica na camada de apresentação.

2. Requisitos

Requisitos funcionais

ID	Descrição	Prioridade	Rota	Status
RF001	Cadastro de pacientes com validação de CPF (algoritmo mod 11)	Alta	/pacientes	✓ Implementado
RF002	Agendar, cancelar e concluir consultas online	Alta	/agendamentos	✓ Implementado
RF003	Visualizar e registrar prontuário eletrônico	Alta	/prontuario	✓ Implementado
RF004	Teleconsulta real via videoconferência (Jitsi Meet)	Média	/telemedicina	✓ Implementado
RF005	Controle de acesso por perfil (RBAC)	Alta	Middleware + Proxy	✓ Implementado
RF006	Emissão de receitas digitais	Média	/receitas	✓ Implementado
RF007	Gestão de profissionais de saúde (CRUD)	Média	/profissionais	✓ Implementado
RF008	Relatórios administrativos com métricas	Média	/relatorios	✓ Implementado
RF009	Dashboard com métricas por perfil	Alta	/dashboard	✓ Implementado
RF010	Notificações de consultas próximas (48h)	Baixa	Layout global	✓ Implementado
RF011	Edição e exclusão de pacientes	Média	/pacientes	✓ Implementado

Requisitos não funcionais

ID	Descrição	Prioridade	Status
RNF001	Interface responsiva (mobile/desktop)	Média	✓ Tailwind + layout responsivo
RNF002	Conformidade LGPD (banner de consentimento, avisos em dados sensíveis)	Alta	✓ Banner + disclaimers
RNF003		Média	✓ Implementado

ID	Descrição	Prioridade	Status
	Usabilidade (componentes shadcn/ui, tooltips, feedbacks visuais)		
RNF004	Acessibilidade (skip-to-content, aria-labels, focus-visible)	Média	✓ WCAG 2.1 AA
RNF005	Tema claro/escuro	Baixa	✓ next-themes
RNF006	Validação de dados no cliente e servidor (Zod)	Alta	✓ Schemas compartilhados
RNF007	Estados de loading, error e not-found	Média	✓ Implementado

Regras de acesso (RBAC)

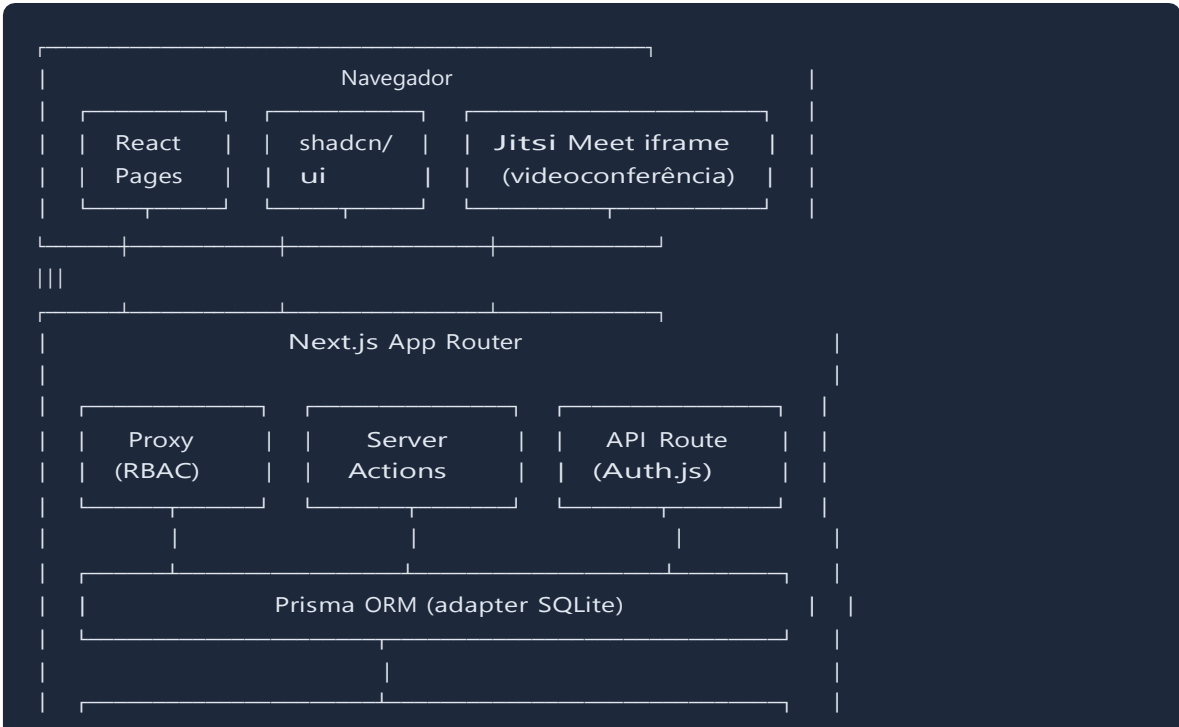
Rota	Admin	Profissional	Paciente
/dashboard	✓	✓	✓
/pacientes	✓	✗	✗
/profissionais	✓	✗	✗
/agendamentos	✓	✓	✓
/prontuario	✓	✓	✓
/receitas	✗	✓	✓
/telemedicina	✗	✓	✓
/relatorios	✓	✗	✗

3. Modelagem e Arquitetura

Stack tecnológica

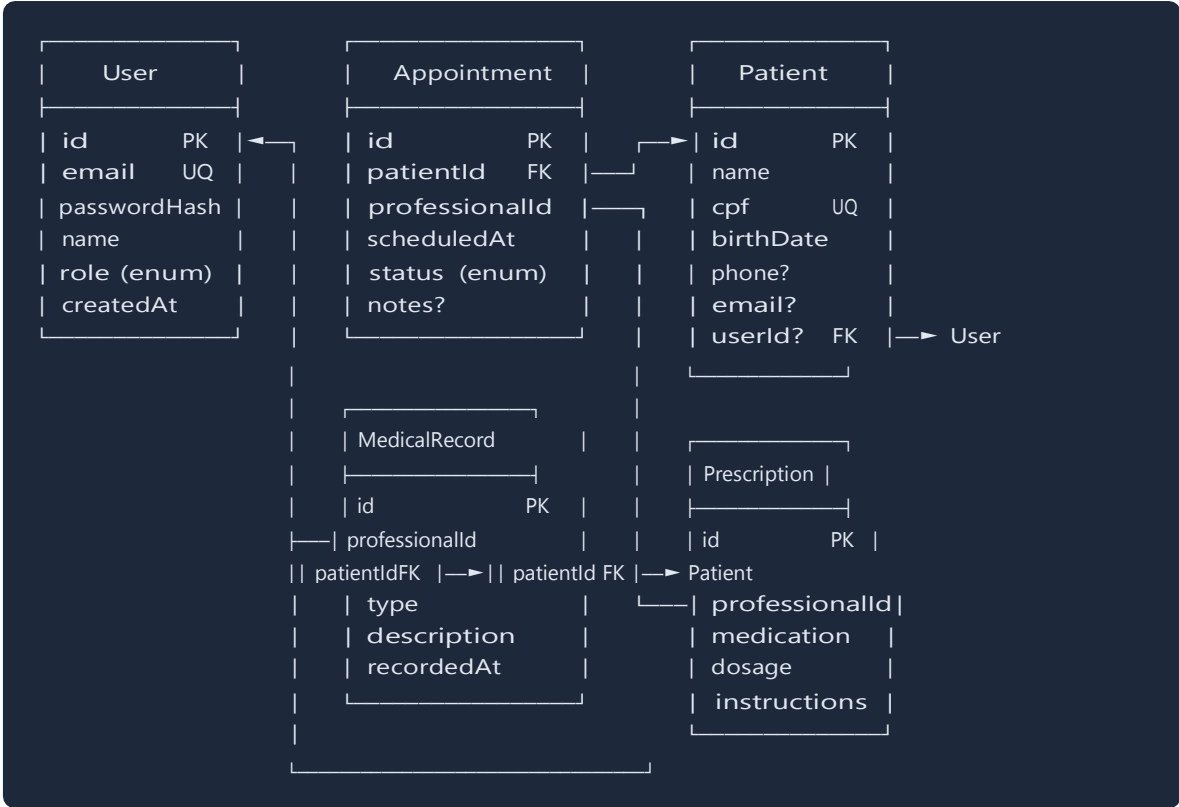
Camada	Tecnologia	Versão
Framework	Next.js (App Router, Turbopack)	16.2
UI	React + TypeScript	19
Estilização	Tailwind CSS	4.x
Componentes	shadcn/ui + @base-ui/react	—
Autenticação	Auth.js (NextAuth v5, Credentials)	5.x
ORM	Prisma (adapter better-sqlite3)	7.x
Banco de dados	SQLite	—
Validação	Zod	3.x
Videoconferência	Jitsi Meet (@jitsi/react-sdk)	—
Testes unitários	Vitest	—
Testes E2E	Playwright	—

Diagrama de arquitetura



```
| | SQLite (dev.db) | |
| |-----| |
| |-----| |
```

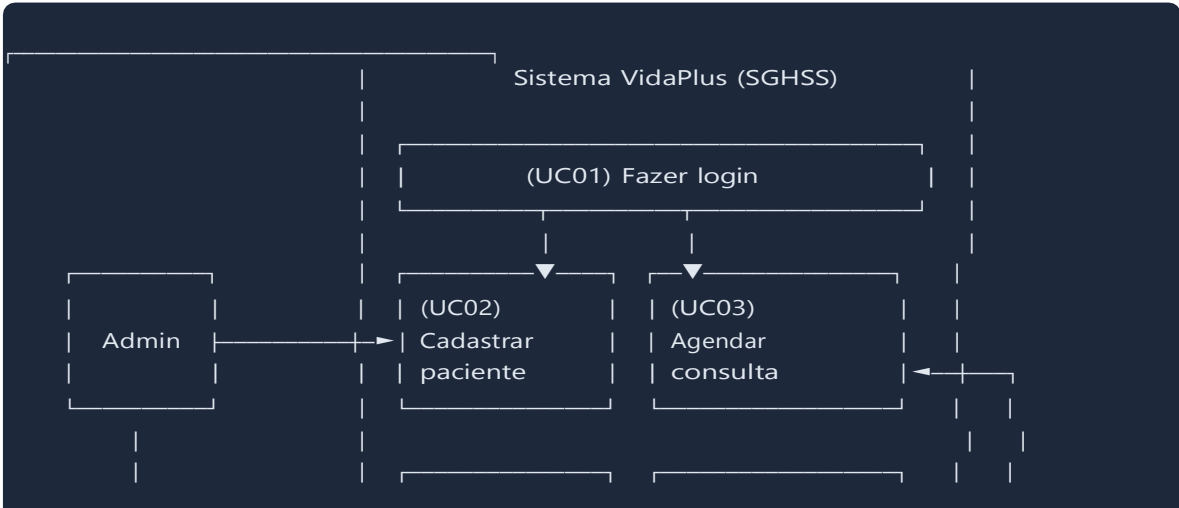
Modelo de dados (Prisma Schema)

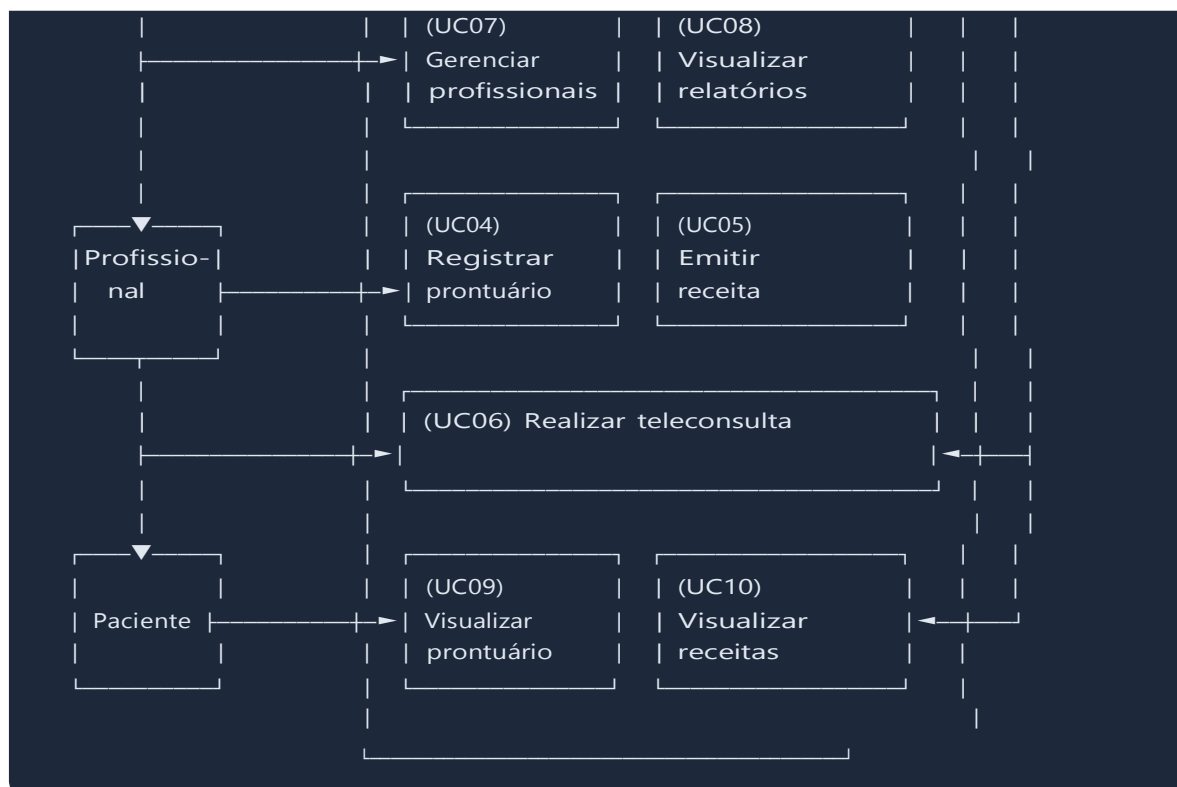


Enums

- **Role:** ADMIN | PATIENT | PROFESSIONAL
- **AppointmentStatus:** SCHEDULED | CANCELLED | COMPLETED

Diagrama de Casos de Uso (UML)





Legenda: Setas indicam associação ator → caso de uso. UC01 é compartilhado por todos os atores. UC06 é compartilhado entre Profissional e Paciente.

Descrição dos Casos de Uso

ID	Caso de uso	Ator(es)	Fluxo principal
UC01	Login por perfil	Todos	Informar e-mail/senha → Auth.js valida → Redireciona ao dashboard por role
UC02	Cadastrar paciente	Admin	Acessar /pacientes → "Novo paciente" → Preencher formulário com CPF → Validação Zod → Server Action cria no banco
UC03	Agendar consulta	Paciente, Admin	Acessar /agendamentos → "Agendar consulta" → Selecionar profissional e data → Confirmação
UC04	Registrar prontuário	Profissional	Acessar /prontuario → Selecionar paciente → "Novo registro" → Tipo + descrição → Salvar
UC05	Emitir receita	Profissional	Acessar /receitas → Selecionar paciente → Medicamento, posologia, instruções → Salvar
UC06	Teleconsulta	Paciente/ Profissional	Acessar /telemedicina → Selecionar consulta agendada → Sala Jitsi Meet é aberta
UC07	Gerenciar profissionais	Admin	Acessar /profissionais → CRUD completo (criar, editar, excluir)
UC08	Visualizar relatórios	Admin	Acessar /relatorios → Métricas agregadas (total pacientes, consultas, status)

ID	Caso de uso	Ator(es)	Fluxo principal
UC09	Visualizar prontuário	Paciente	Acessar /prontuario → Ver histórico clínico próprio (somente leitura)
UC10	Visualizar receitas	Paciente	Acessar /receitas → Ver receitas prescritas para si (somente leitura)

Estrutura de pastas

```

vida-plus/
├── app/
│   ├── (dashboard)/ # Layout autenticado
│   │   ├── agendamentos/ # Consultas e agendamentos
│   │   ├── dashboard/ # Painel com métricas
│   │   ├── pacientes/ # CRUD de pacientes
│   │   ├── profissionais/ # CRUD de profissionais
│   │   ├── prontuario/ # Prontuário eletrônico
│   │   ├── receitas/ # Receitas digitais
│   │   ├── relatorios/ # Relatórios administrativos
│   │   ├── telemedicina/ # Videoconsulta Jitsi
│   │   ├── layout.tsx # Shell + notificações
│   │   ├── loading.tsx # Skeleton de carregamento
│   │   ├── error.tsx # Tratamento de erros
│   │   └── not-found.tsx # Página 404
│   ├── actions/ # Server Actions (CRUD)
│   ├── api/auth/ # NextAuth route handler
│   ├── login/ # Página de login
│   └── layout.tsx # Root layout + providers
├── components/
│   ├── ui/ # Componentes shadcn/ui
│   ├── telemedicina/ # Jitsi room + mock video
│   └── *.tsx # Formulários, botões, dialogs # Auth,
├── lib/ Prisma, validadores Zod # Schema + seed
├── prisma/ # Testes Playwright
├── e2e/ # Testes unitários Vitest # Este relatório
├── __tests__/
└── docs/report/

```

4. Implementação

Repositório

GitHub: <https://github.com/icarobteles/vida-plus>

Deploy

Vercel: <https://vida-plus-seven.vercel.app>

Credenciais de demonstração

Perfil	E-mail	Senha
Administrador	admin@vida.plus	123456
Paciente	paciente@vida.plus	123456
Profissional	medico@vida.plus	123456

Módulos implementados

4.1 Autenticação e RBAC

- Auth.js v5 com provider `Credentials` e sessão JWT
- Login com e-mail/senha, validação via `bcryptjs`
- Middleware (Proxy) intercepta rotas e redireciona usuários sem permissão
- Sessão estendida com `role` e `patientId` para controle granular

4.2 Dashboard (`/dashboard`)

Painel personalizado por perfil:

- **Admin:** total de pacientes, profissionais, consultas agendadas e concluídas
- **Profissional:** consultas do dia, total de prontuários registrados
- **Paciente:** próximas consultas, total de registros no prontuário

4.3 Pacientes (`/pacientes`) — Admin

- Listagem em tabela com busca
- Cadastro via dialog com validação de CPF (algoritmo módulo 11 via Zod)
- Edição in-place via dialog
- Exclusão com dialog de confirmação estilizado
- Ações em icon buttons com tooltips

4.4 Profissionais (/profissionais) — Admin

- CRUD completo via dialog (criar, editar, excluir)
- Hash de senha com `bcryptjs` no Server Action
- Edição permite atualizar nome/e-mail sem alterar senha

4.5 Agendamentos (/agendamentos)

- **Paciente:** agenda consultas selecionando profissional e data/hora
- **Profissional:** visualiza sua agenda, conclui ou cancela consultas
- **Admin:** visão completa de todos os agendamentos
- Status: `SCHEDULED` → `COMPLETED` ou `CANCELLED`
- Link direto para teleconsulta em consultas agendadas

4.6 Prontuário eletrônico (/prontuario)

- **Profissional:** seleciona paciente → registra tipo + descrição clínica
- **Paciente:** visualiza seu histórico (somente leitura)
- Página de detalhe por paciente (/prontuario/[id])

4.7 Receitas digitais (/receitas)

- **Profissional:** selecionar paciente → medicamento, posologia e instruções
- **Paciente:** visualiza receitas emitidas para si (somente leitura)
- Histórico de receitas emitidas por paciente

4.8 Teleconsulta (/telemedicina)

- Integração real com **Jitsi Meet** via `@jitsi/react-sdk`
- Sala de vídeo criada automaticamente a partir do ID do agendamento
- Nome da sala legível: `VidaPlusTeleconsulta{hash}`
- Título da sala: Teleconsulta — Paciente X
- Saída limpa: listener `videoConferenceLeft` encerra o componente imediatamente

4.9 Relatórios (/relatorios) — Admin

- Métricas agregadas: total de pacientes, profissionais, consultas
- Distribuição por status (agendadas, concluídas, canceladas)
- Cards visuais com contadores

4.10 Notificações

- Popover no header com consultas nas próximas 48 horas
- Exibe tempo restante (ex.: "em 23h 15min")
- Badge numérico indicando quantidade de notificações

4.11 LGPD

- **Banner de consentimento** na primeira visita (persistido via `localStorage`)
- **Aviso no login:** informação sobre coleta de dados para autenticação

- **Aviso no prontuário:** alerta sobre dados clínicos sensíveis

4.12 Acessibilidade (WCAG)

- **Skip-to-content:** link de atalho para o conteúdo principal
- **Aria-labels:** em todos os controles interativos
- **Focus-visible:** estilo de foco visível para navegação por teclado
- **Tooltips:** em todos os icon buttons com descrição da ação
- **Dark mode:** tema claro/escuro via `next-themes`

4.13 UX e estados

- **Loading:** skeleton animado durante carregamento de páginas
- **Error:** boundary de erro com mensagem amigável e botão de retry
- **Not-found:** página 404 estilizada com link para o dashboard
- **Toasts:** feedback visual via `sonner` para ações bem-sucedidas ou erros
- **Dialogs de confirmação:** para ações destrutivas (excluir, cancelar)

Fluxo de desenvolvimento

O projeto foi desenvolvido em **6 entregas incrementais**, cada uma em branch dedicada:

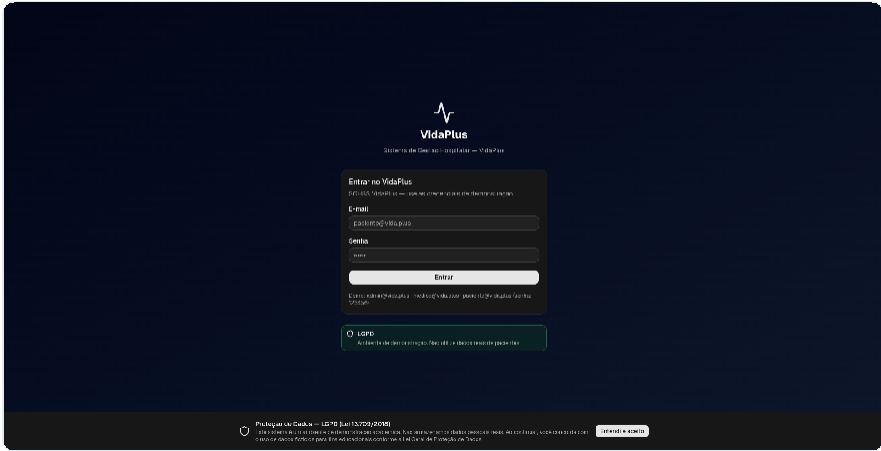
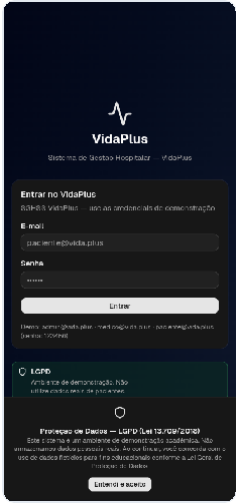
#	Branch	Escopo
1	fix/rbac-e-correcoes	Correção de RBAC, métricas no dashboard, ThemeProvider
2	feat/profissional-completo	Agenda, prontuário e receitas para profissional
3	feat/telemedicina-jitsi	Teleconsulta real com Jitsi Meet
4	feat/admin-gestao	CRUD profissionais, edição de pacientes, relatórios
5	feat/notificacoes-lgpd-a11y	Notificações, LGPD, acessibilidade, loading states
6	test/cobertura-completa	Testes unitários e E2E expandidos, seed setup/teardown
7	feat/melhorias-finais	Receitas para pacientes, cobertura E2E expandida, docs
8	fix/seguranca-e-responsividade	Correção IDOR, try/catch, responsividade mobile, aria-labels

Cada entrega seguiu **Conventional Commits** em português com commits atômicos.

Screenshots das telas

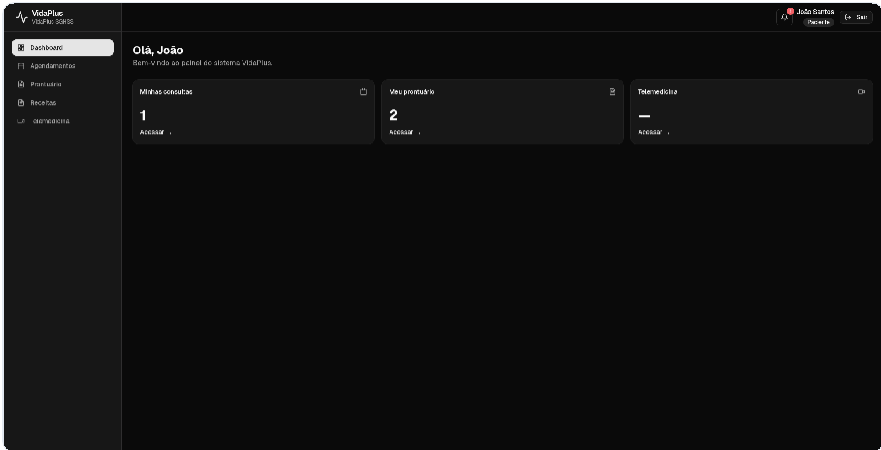
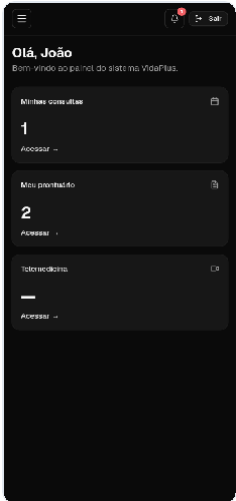
As capturas a seguir cobrem **todas as telas do sistema** nos três perfis de usuário, em versão **desktop** e **mobile**, demonstrando a responsividade da aplicação.

Autenticação

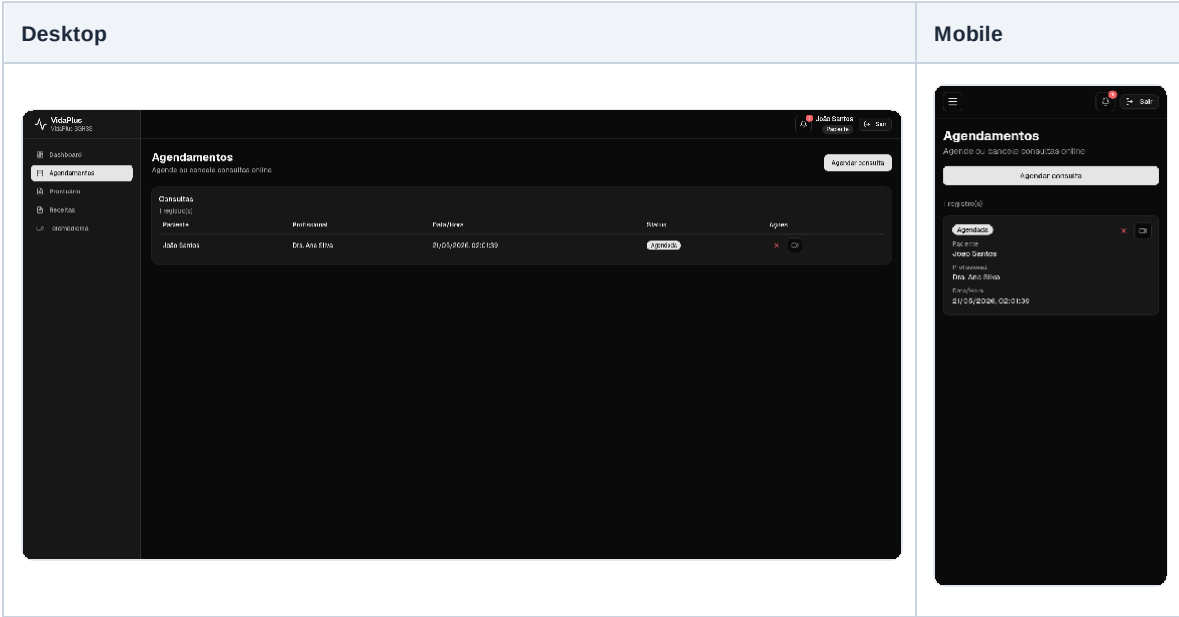
Desktop	Mobile
	

Tela de login com credenciais de demonstração e aviso LGPD.

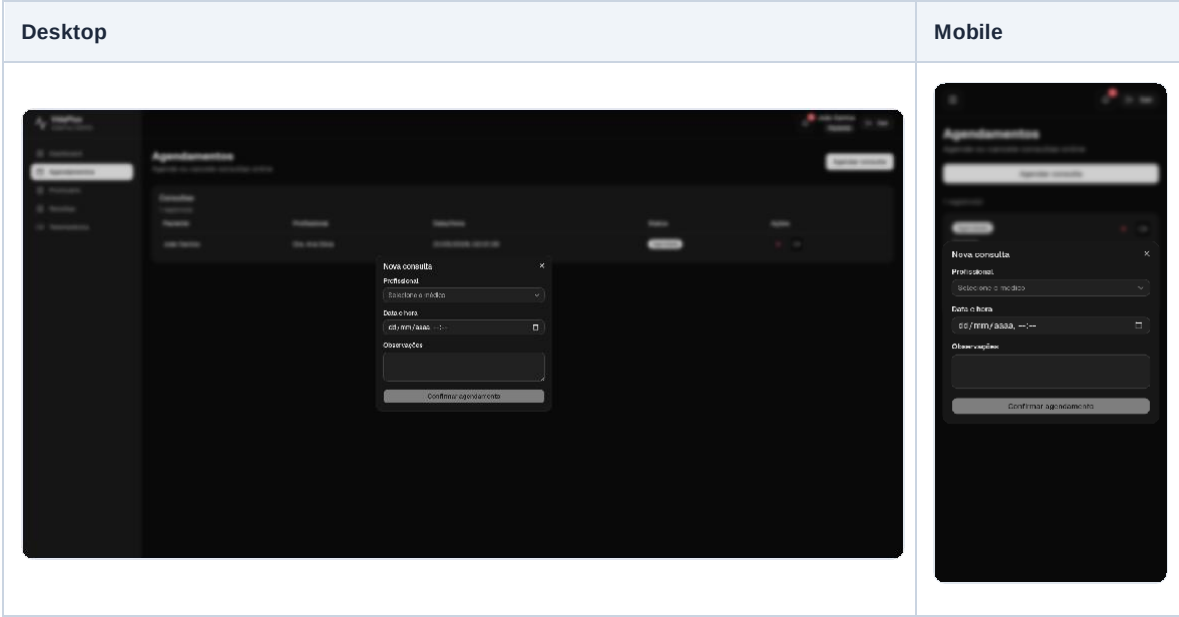
Visão do Paciente

Desktop	Mobile
	

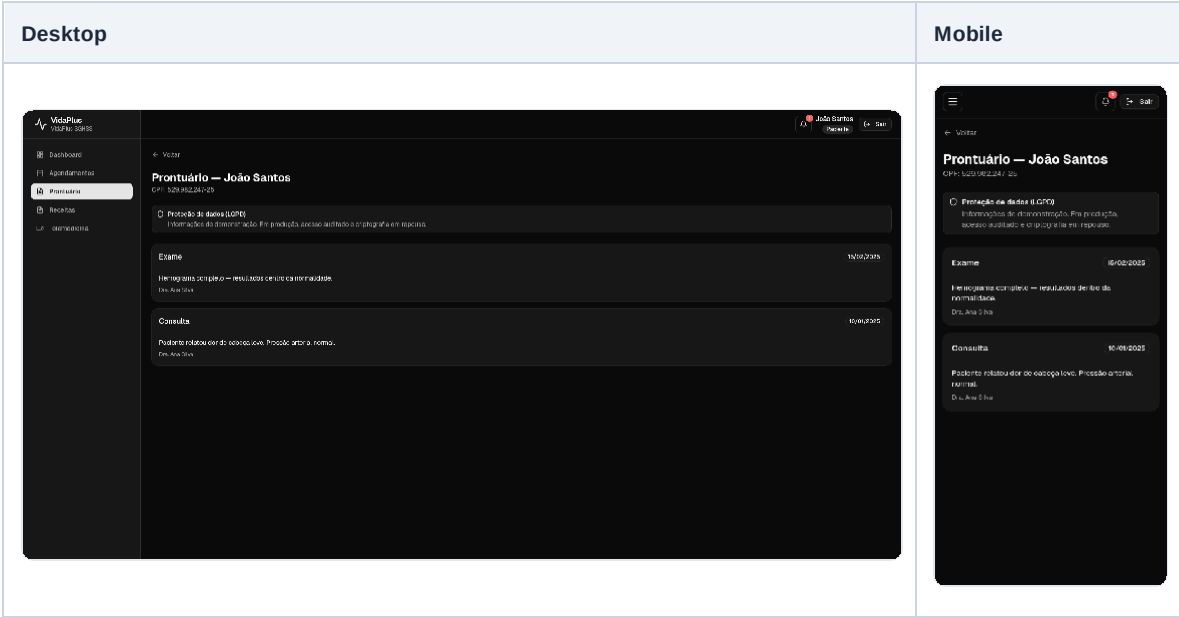
Dashboard do paciente com métricas pessoais.



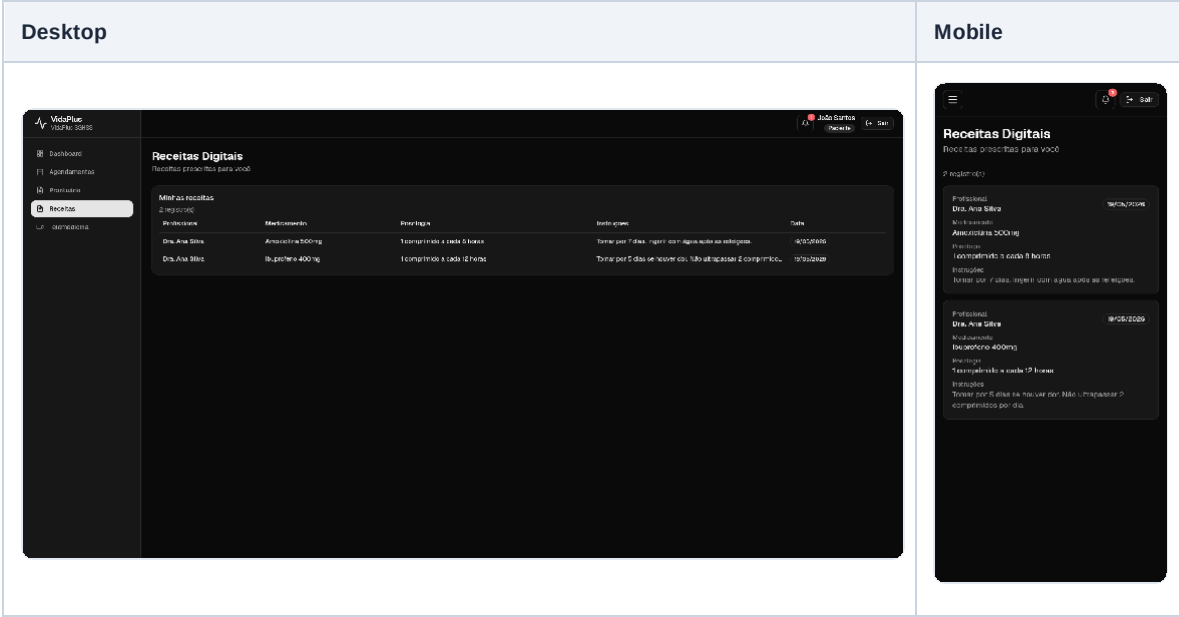
Lista de consultas agendadas do paciente.



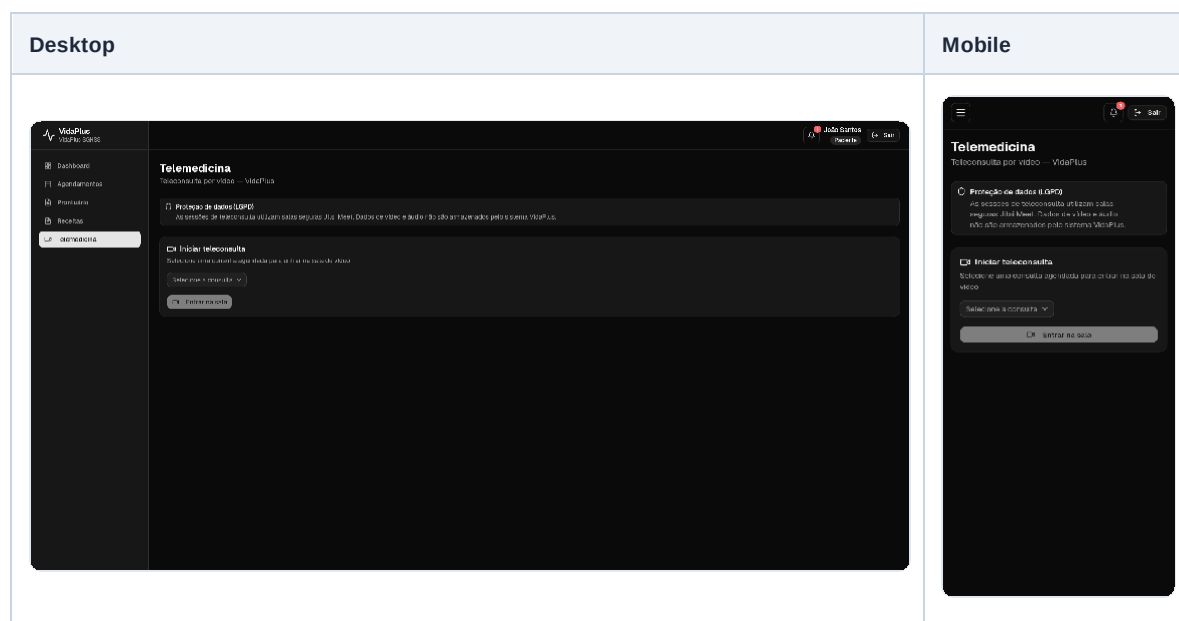
Formulário de agendamento de nova consulta.



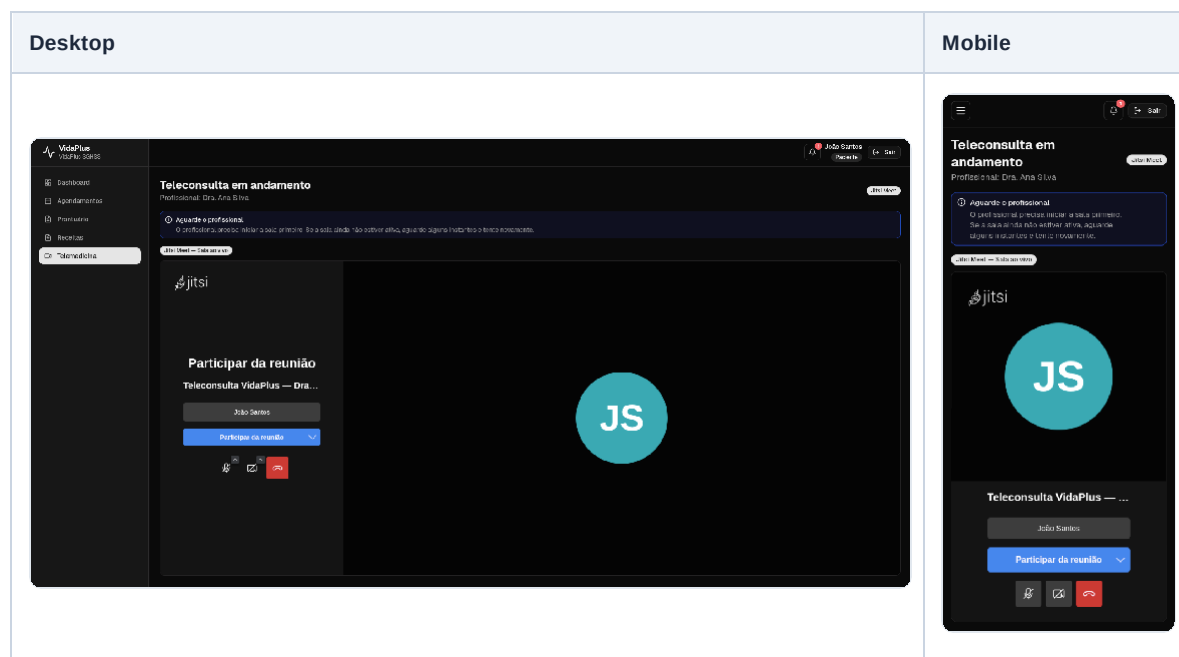
Histórico clínico do paciente com registros e aviso LGPD.



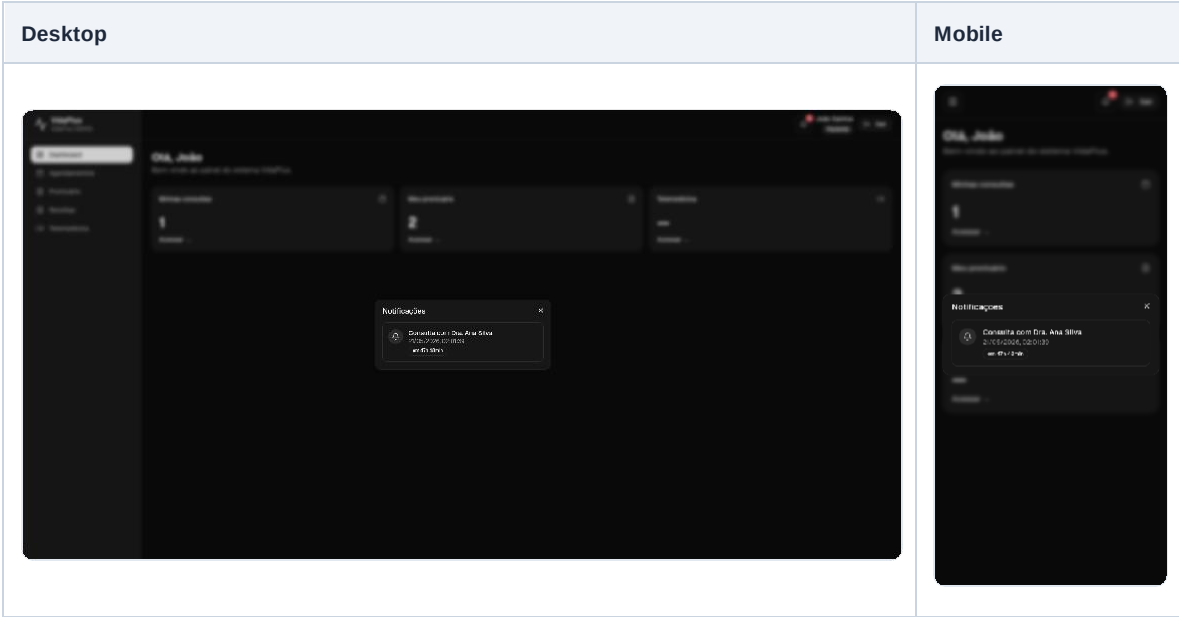
Receitas prescritas para o paciente (somente leitura).



Tela de teleconsulta com seleção de consulta agendada.

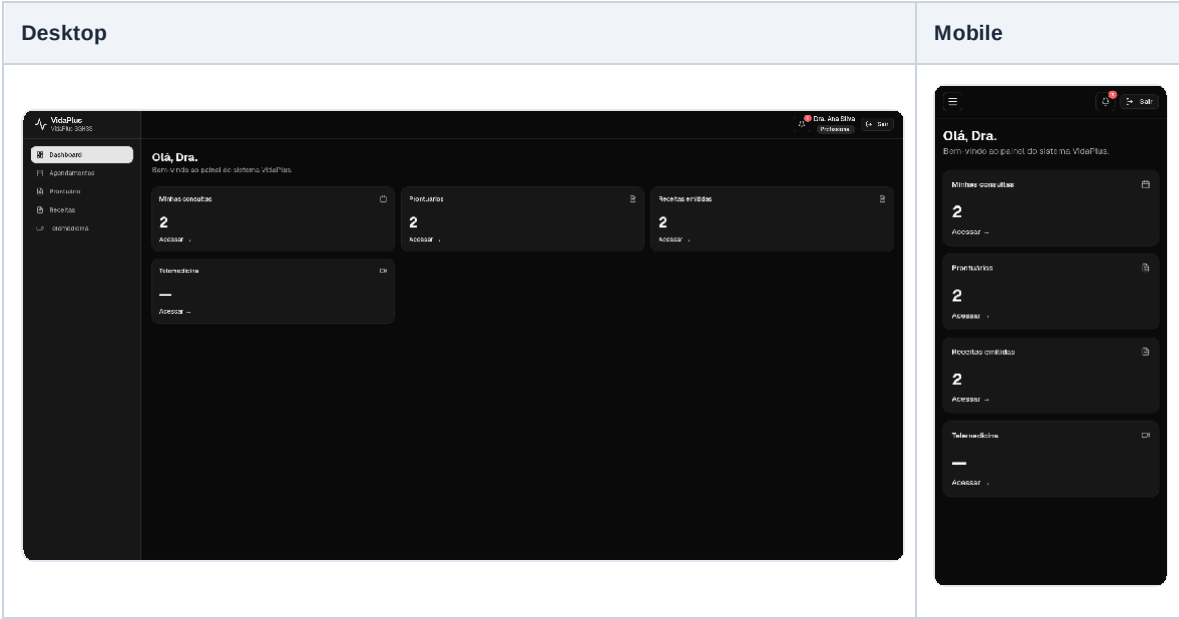


Sala de videoconsulta Jitsi Meet aguardando início.

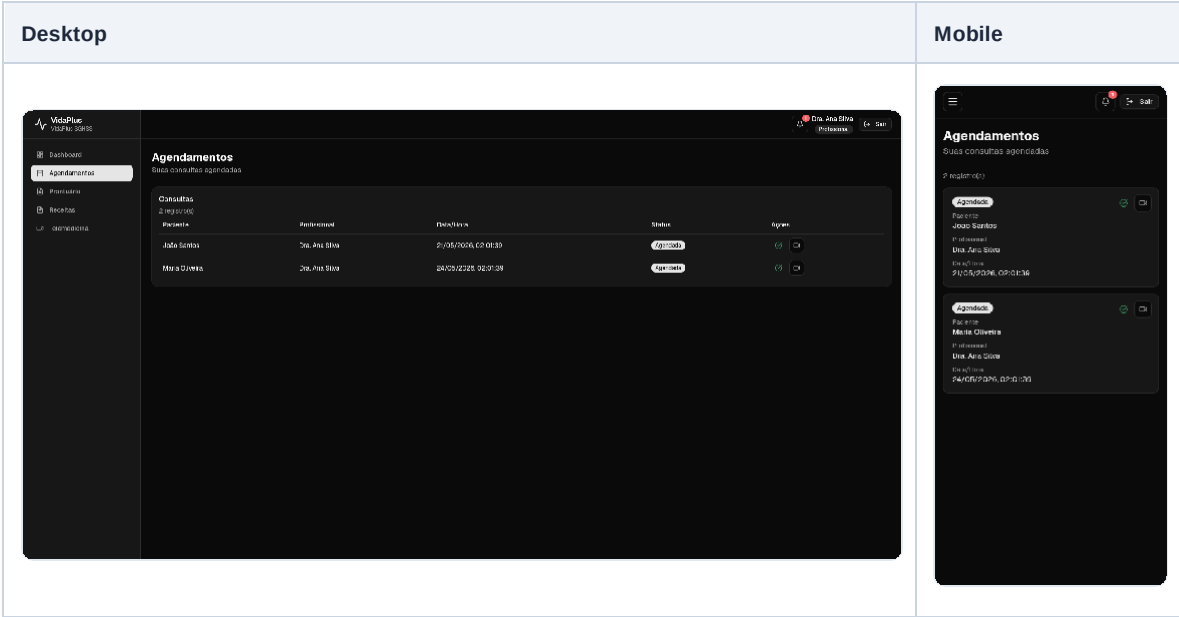


Popover de notificações com consultas nas próximas 48 horas.

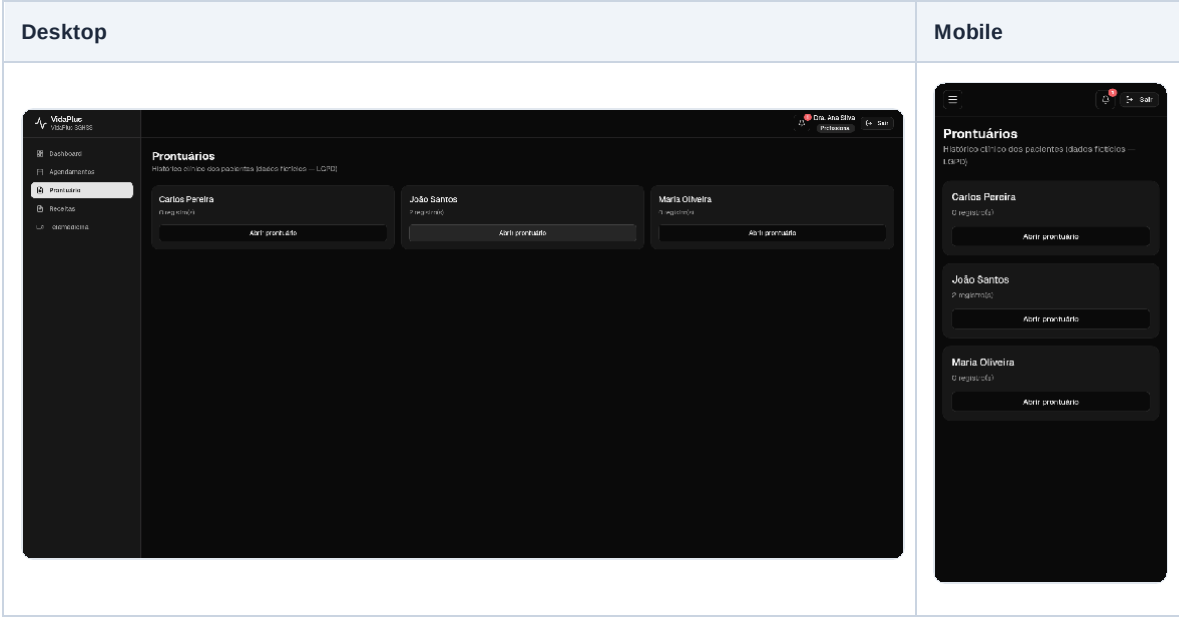
Visão do Profissional



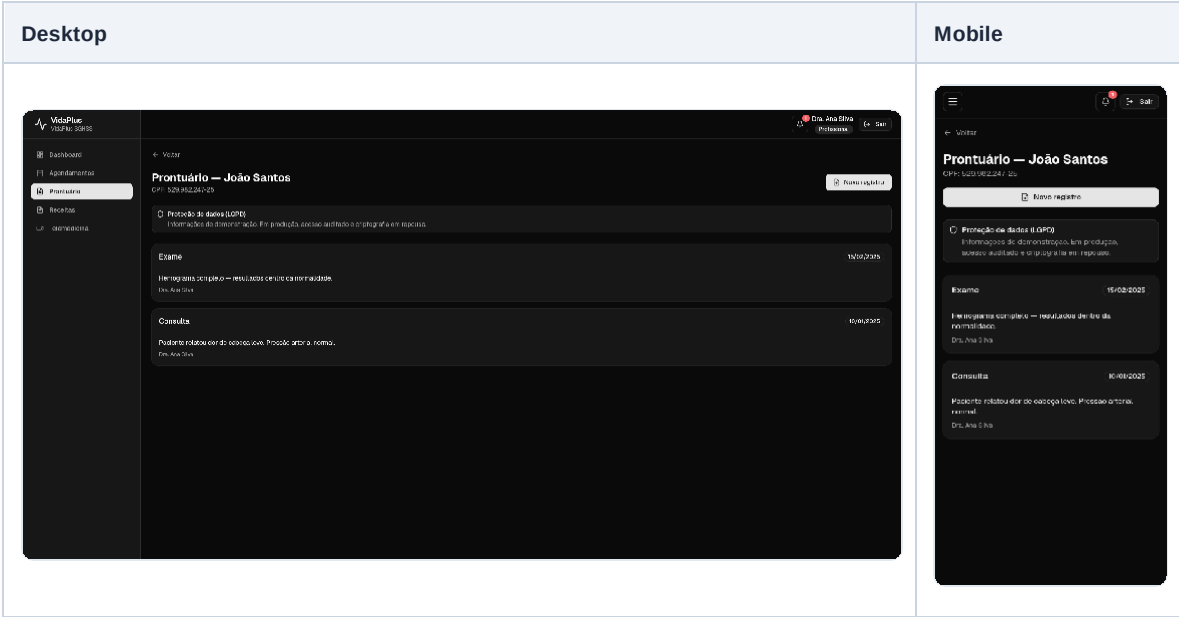
Dashboard do profissional com suas consultas e prontuários.



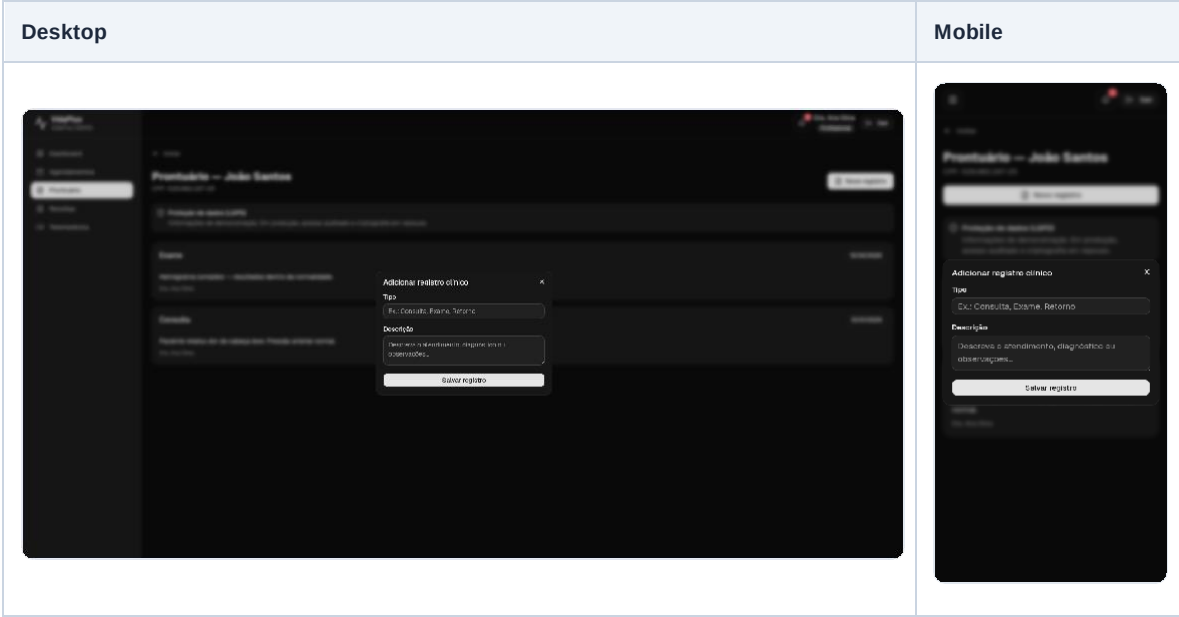
Agenda do profissional com ações de concluir e link para teleconsulta.



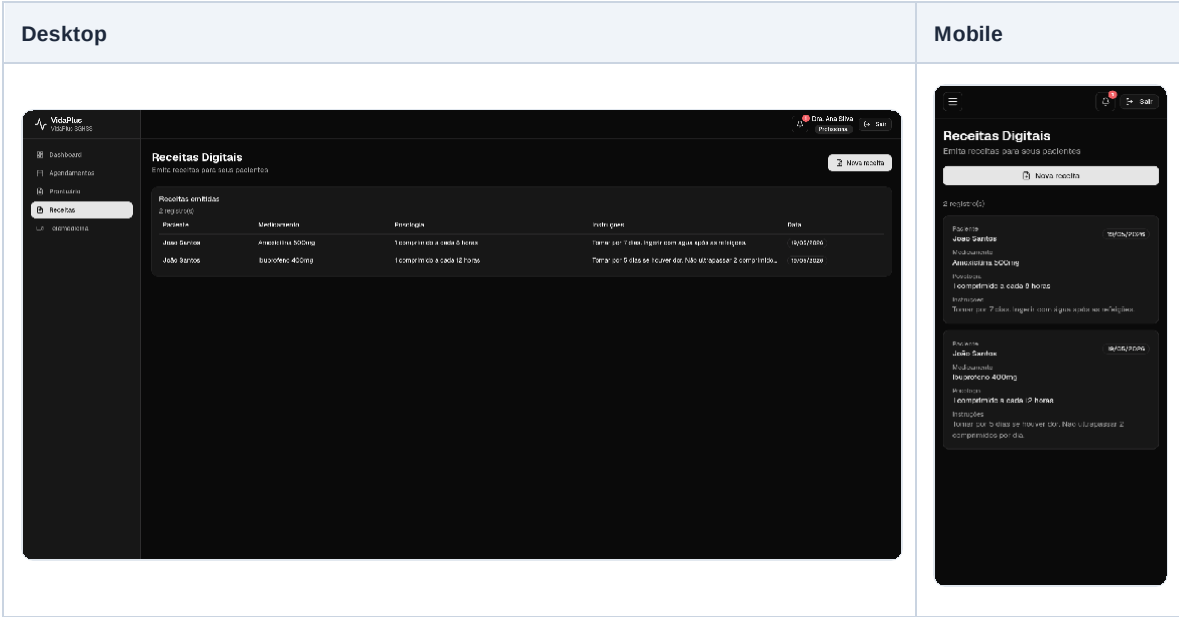
Lista de pacientes para acesso ao prontuário.



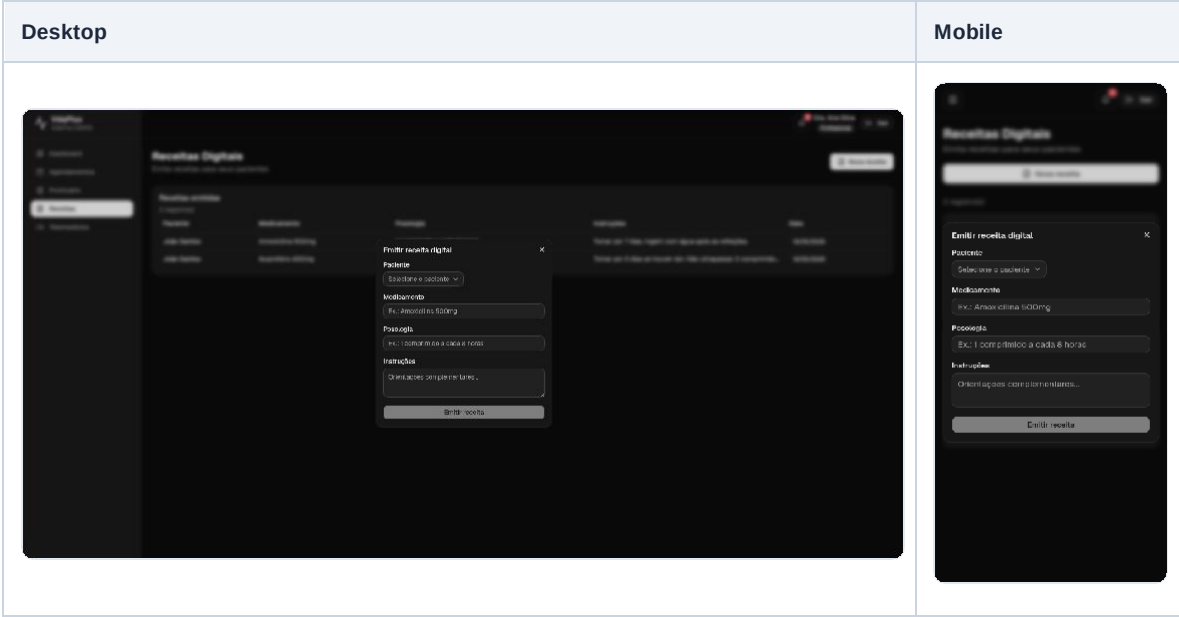
Detalhe do prontuário de um paciente específico.



Formulário para registrar novo evento no prontuário.

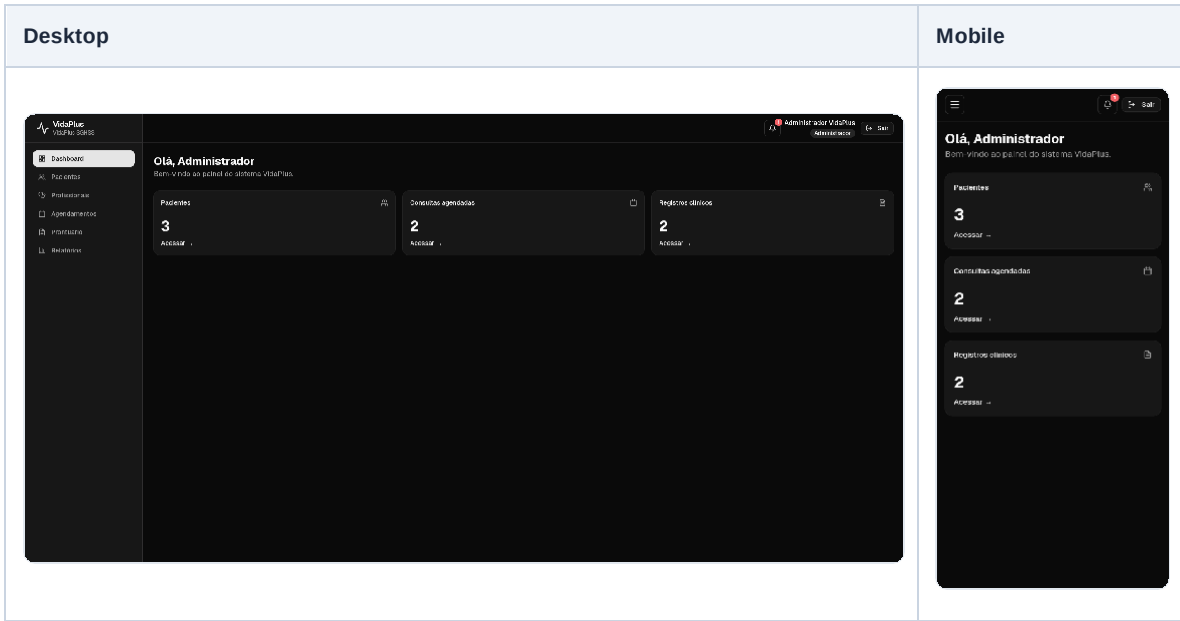


Lista de receitas emitidas pelo profissional.

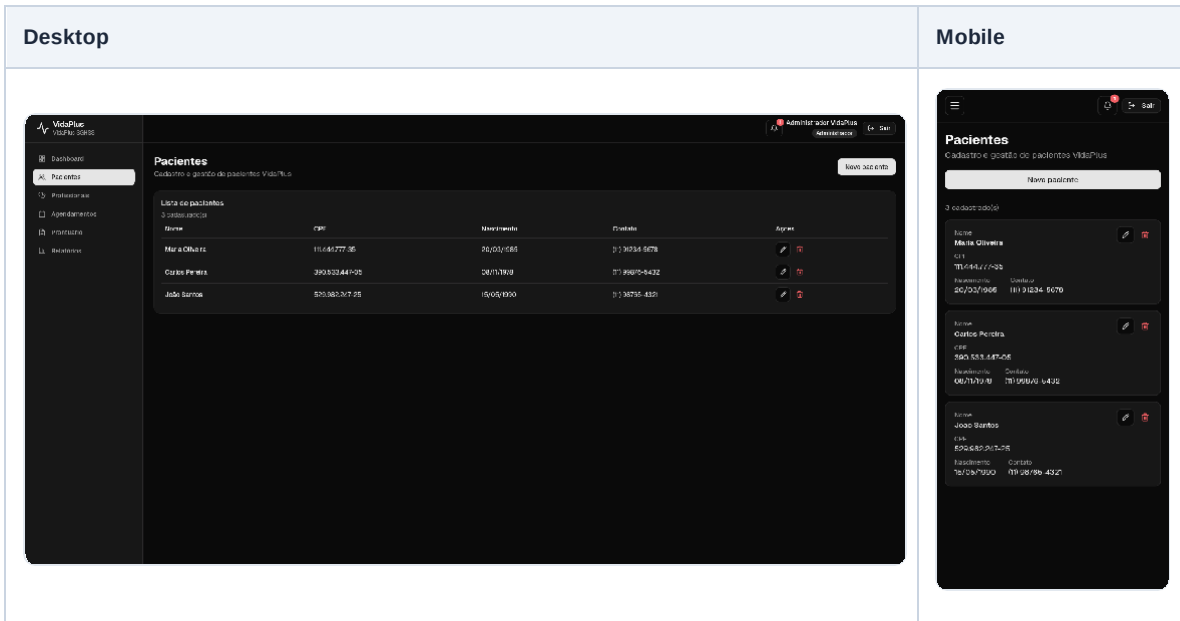


Formulário de emissão de receita digital.

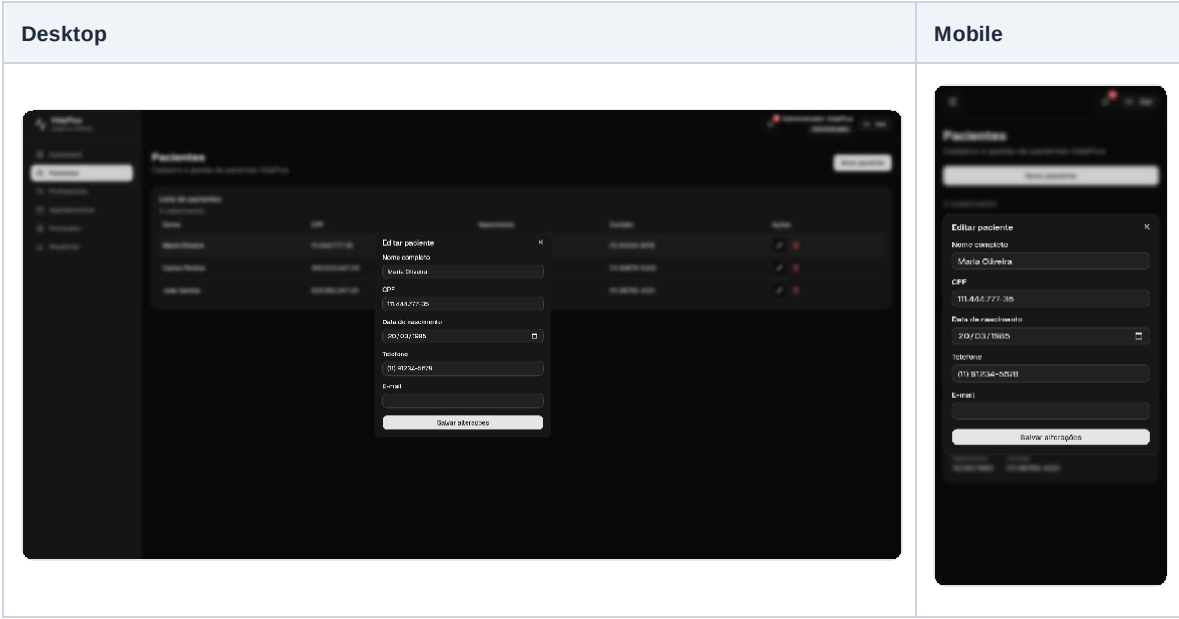
Visão do Administrador



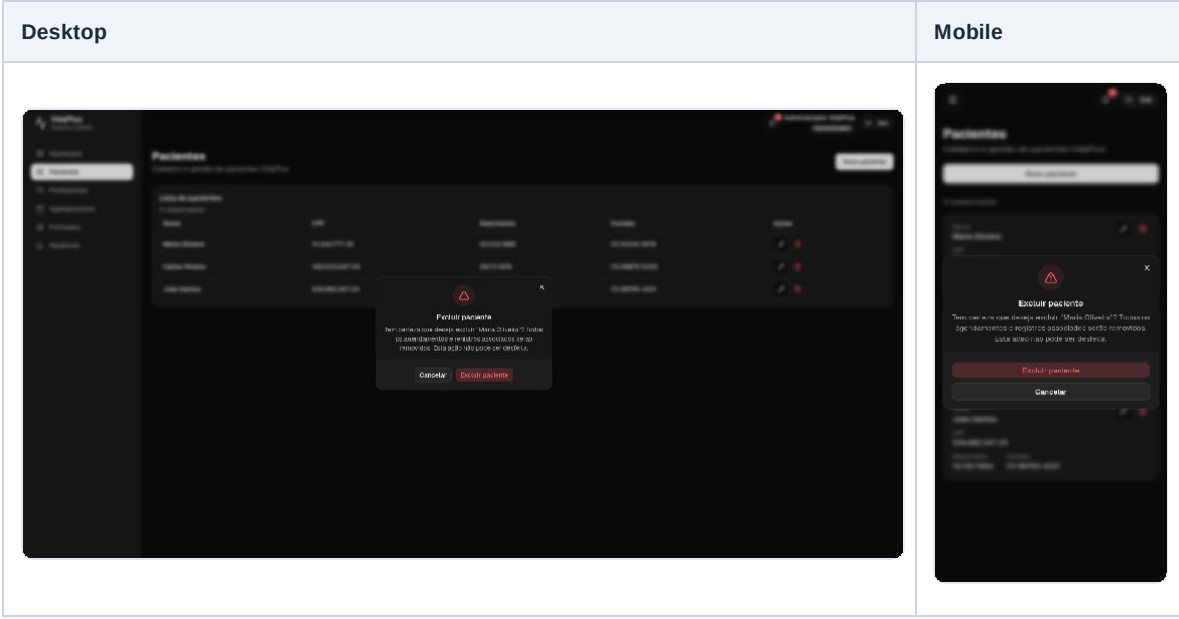
Dashboard do administrador com métricas gerais do sistema.



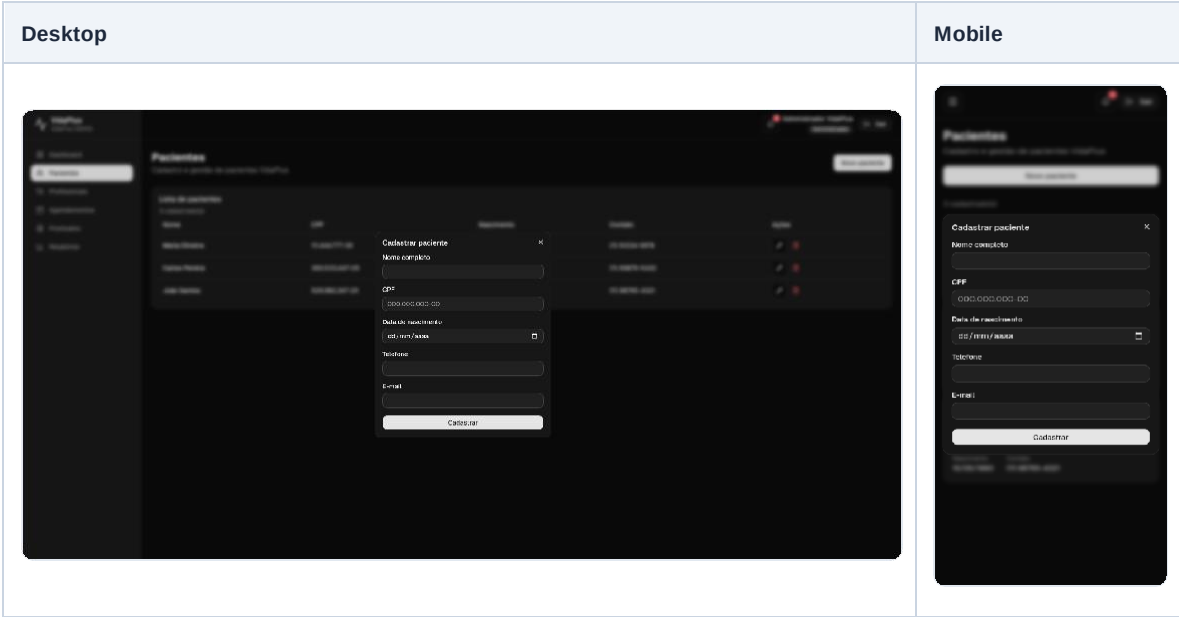
Listagem de pacientes com ações de editar e excluir.



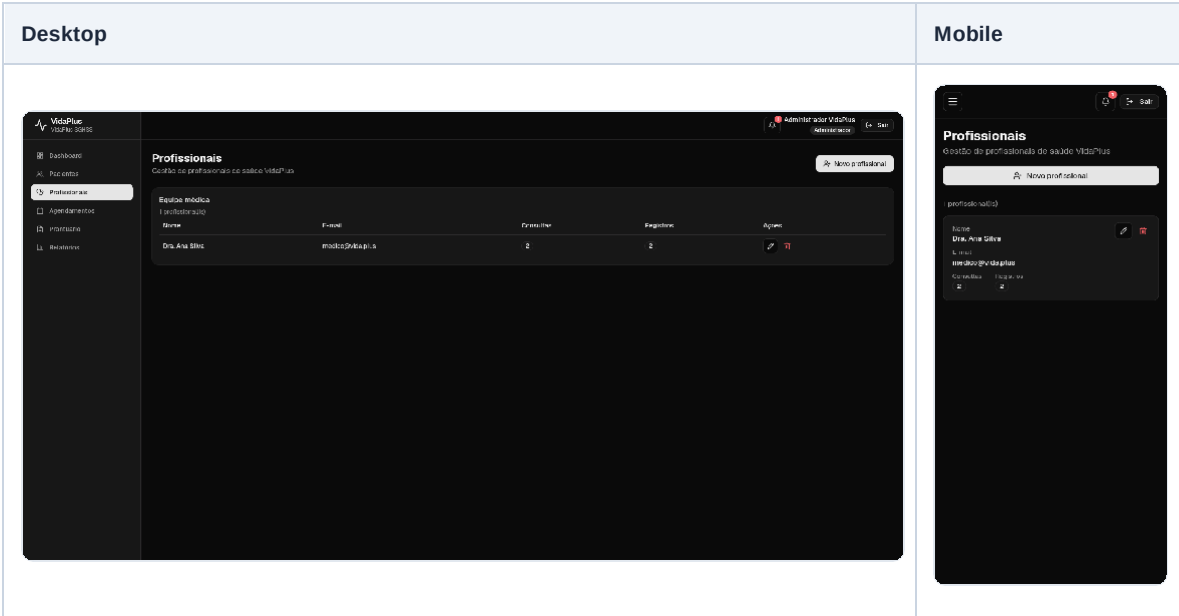
Dialog de edição de dados do paciente.



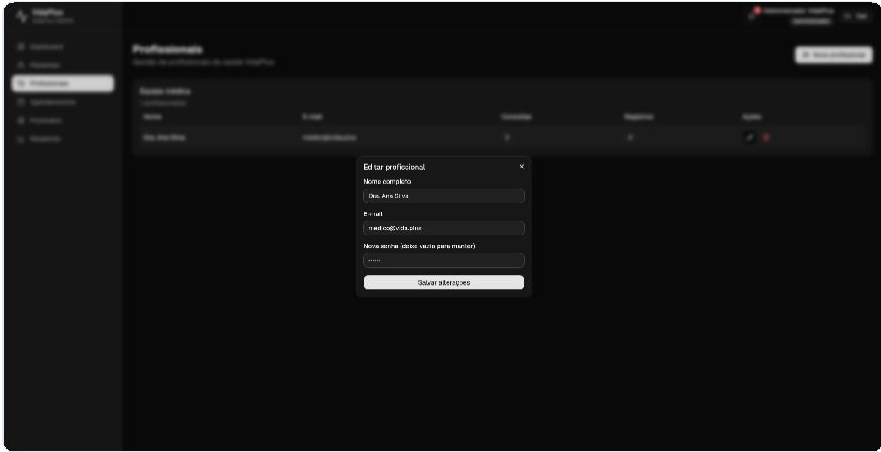
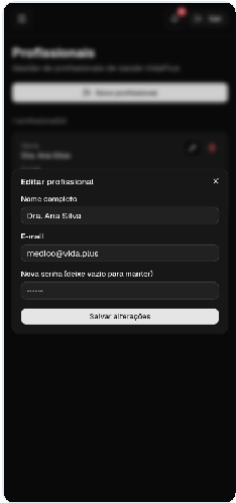
Confirmação de exclusão de paciente.



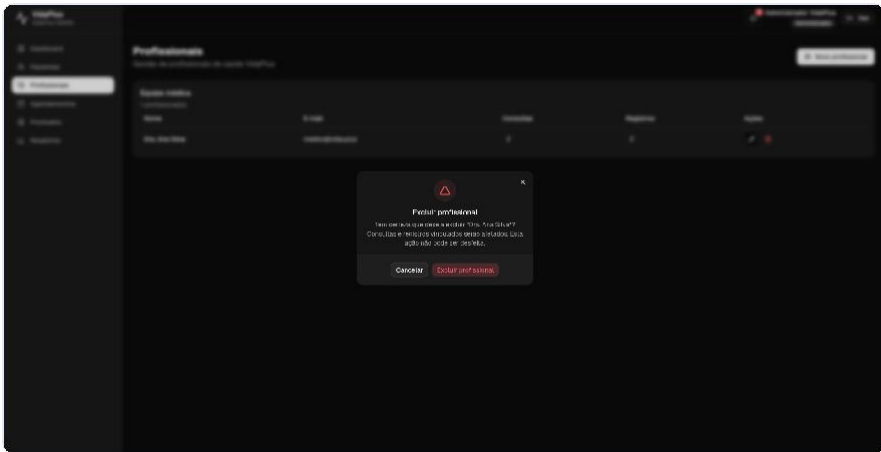
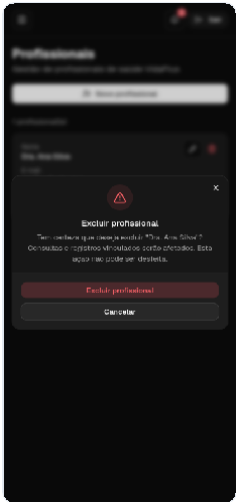
Formulário de cadastro de novo paciente com validação de CPF.



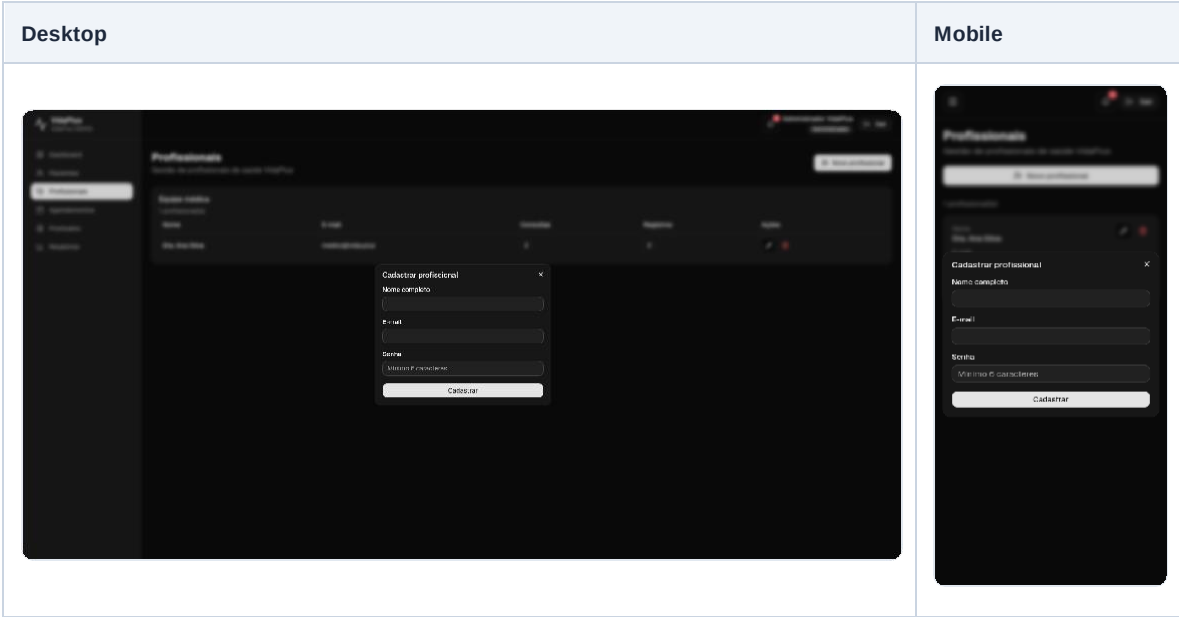
Gestão de profissionais de saúde.

Desktop	Mobile
	

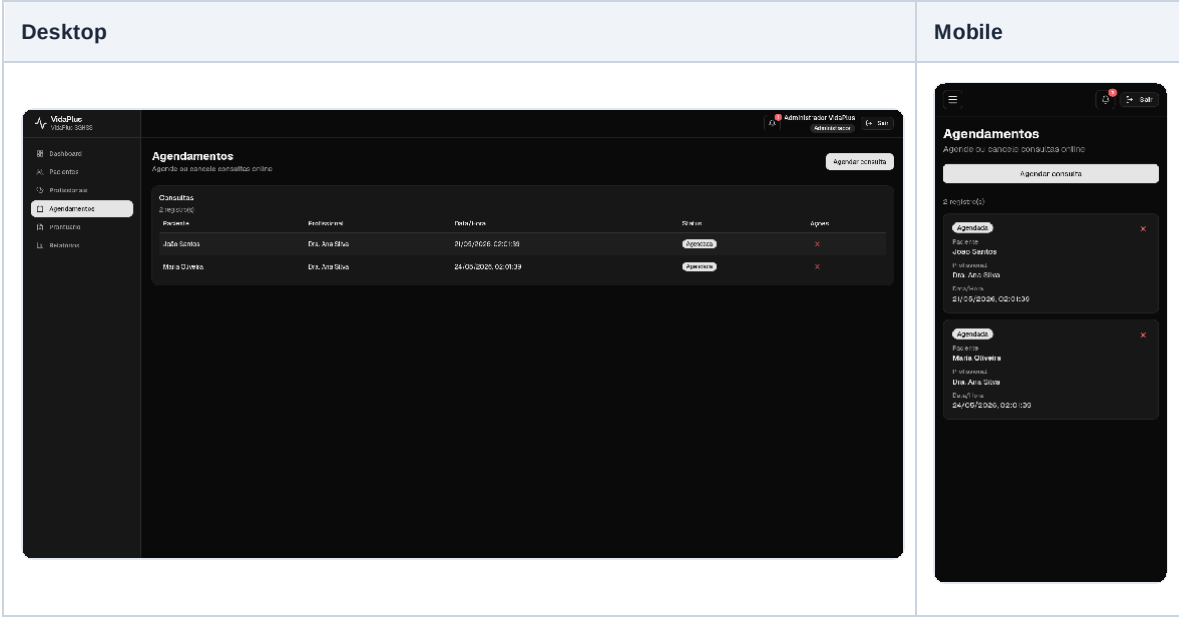
Dialog de edição de profissional.

Desktop	Mobile
	

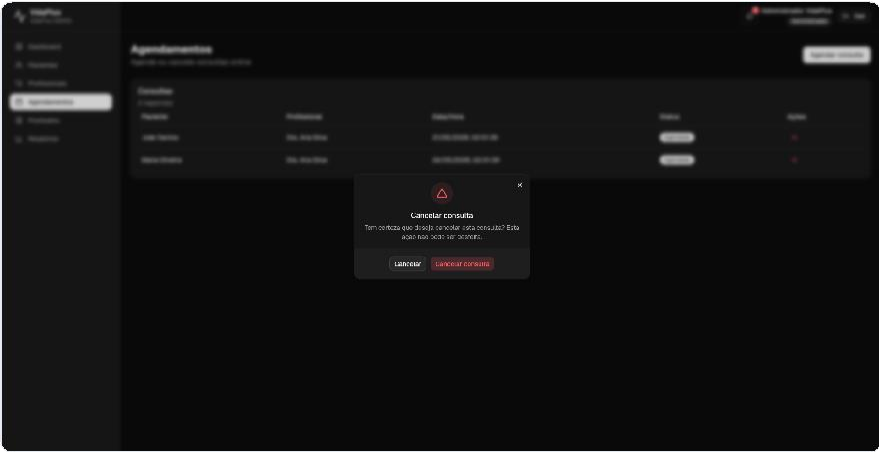
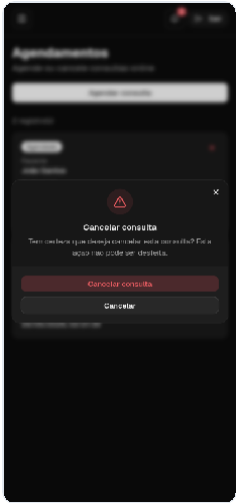
Confirmação de exclusão de profissional.



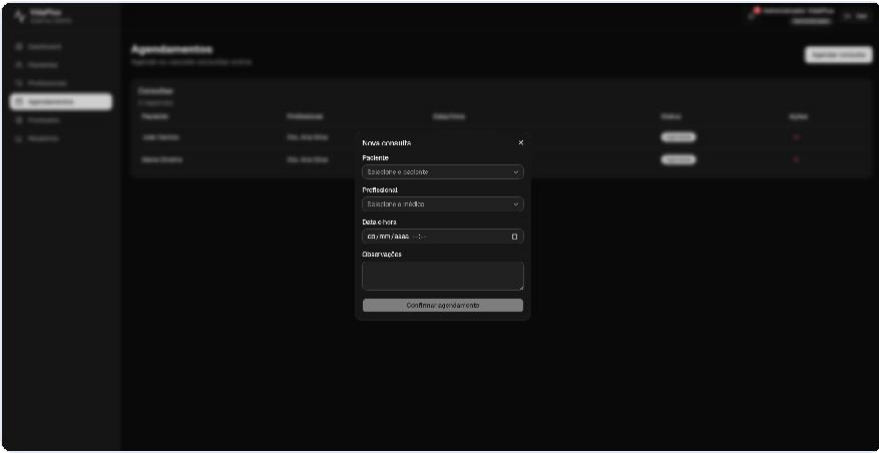
Formulário de cadastro de novo profissional.



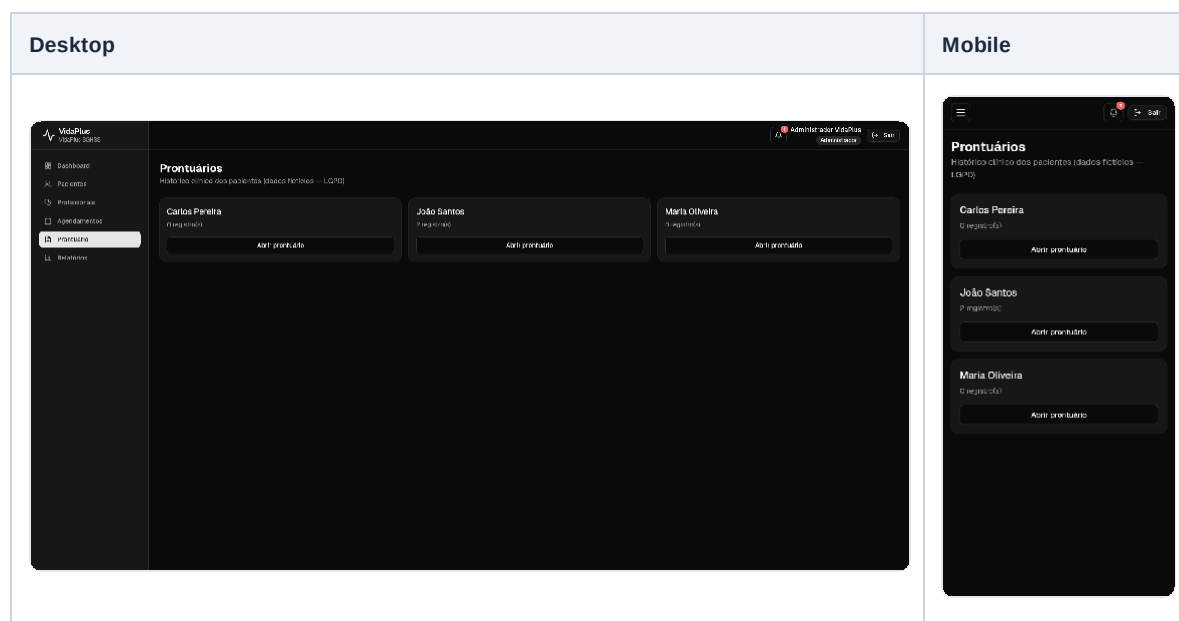
Visão completa de todos os agendamentos.

Desktop	Mobile
	

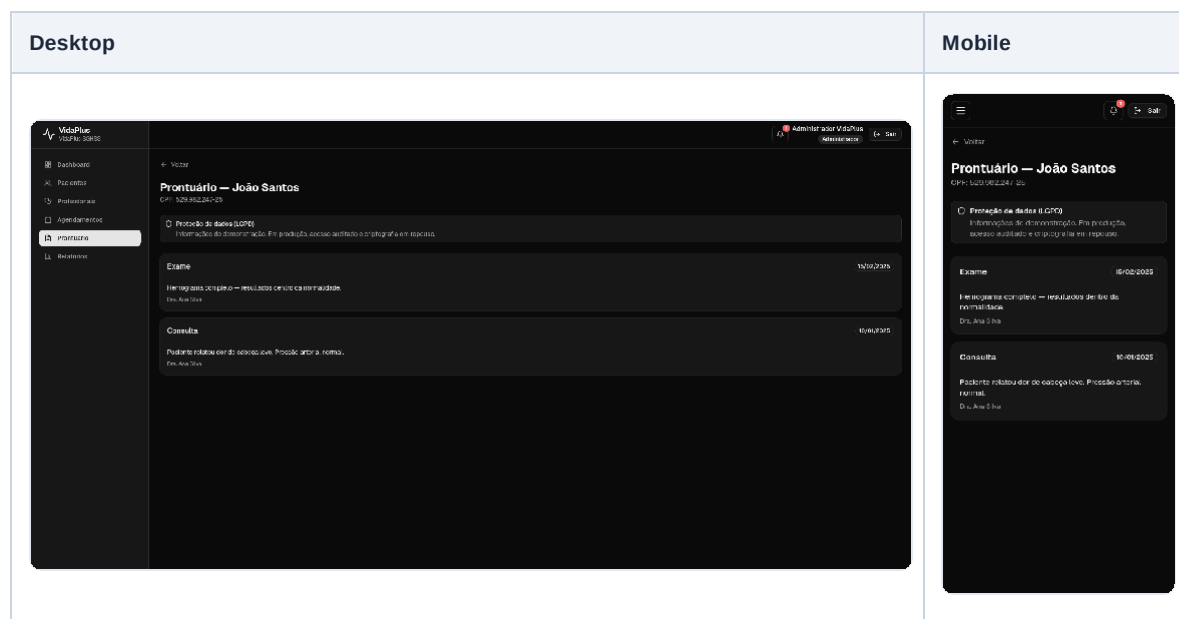
Confirmação de cancelamento de consulta.

Desktop	Mobile
	

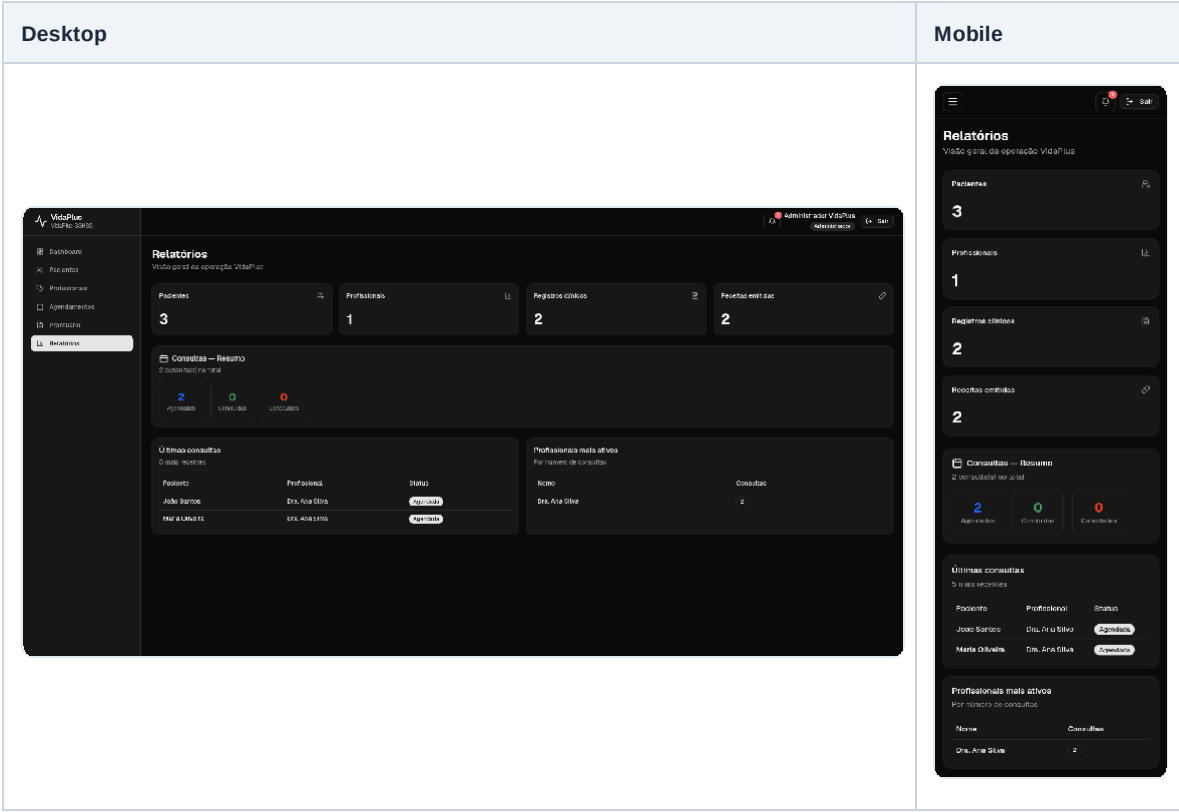
Formulário de agendamento de consulta.



Prontuários dos pacientes com cards por paciente.



Detalhe do prontuário de um paciente (visão admin).



Painel de relatórios com métricas agregadas e distribuição por status.

5. Plano de Testes

Ferramentas

Ferramenta	Tipo	Uso
Vitest	Unitário	Validação de schemas Zod, lógica de CPF
Playwright	E2E (end-to-end)	Fluxos completos no navegador Chromium

Testes unitários (Vitest) — 32 testes

Arquivo: `_tests_/validators.test.ts`

Suite	Casos	Exemplos
<code>isValidCpf</code>	5	CPF válido, dígitos iguais, dígito verificador errado, menos de 11 dígitos, com pontuação
<code>patientSchema</code>	8	Dados válidos, campos opcionais, nome curto, CPF vazio/inválido, data vazia, e-mail inválido
<code>appointmentSchema</code>	4	Agendamento válido, com observações, sem profissional, sem data
<code>medicalRecordSchema</code>	3	Registro válido, tipo vazio, descrição curta
<code>prescriptionSchema</code>	4	Receita válida, sem medicamento, sem posologia, sem instruções
<code>professionalSchema</code>	5	Profissional válido, sem senha (edição), nome curto, e-mail inválido, senha curta
<code>loginSchema</code>	3	Login válido, e-mail inválido, senha vazia

Execução

```
npm test
```

Resultado

```
_tests_/validators.test.ts (32 tests)

Test Files  1 passed (1)
Tests      32 passed (32) Duration  155ms
```

Testes E2E (Playwright) — 20 testes

Configuração

- **Porta:** 3000 (reutiliza o dev server do desenvolvedor)
- **globalSetup:** re-seed do banco antes dos testes (dados previsíveis)
- **globalTeardown:** re-seed após os testes (restaura dados de desenvolvimento)
- **Workers:** 4 paralelos
- **Retry:** 1 tentativa em caso de falha (2 em CI)

Catálogo de testes

Arquivo	ID	Descrição	Perfil
login-admin.spec.ts	CT001	Login do admin redireciona para o dashboard	Admin
login-admin.spec.ts	CT002	Admin vê links de pacientes, profissionais e relatórios	Admin
login-profissional.spec.ts	CT003	Login do profissional redireciona para o dashboard	Profissional
login-profissional.spec.ts	CT004	Profissional vê links de agendamentos, prontuário, receitas e telemedicina	Profissional
login-paciente.spec.ts	CT005	Login do paciente redireciona para o dashboard	Paciente
cadastro-paciente.spec.ts	CT006	Admin cadastra paciente com CPF válido	Admin
agendamento.spec.ts	CT007	Paciente agenda consulta	Paciente
rbac.spec.ts	CT008	Paciente não acessa /pacientes (redirecionado)	Paciente
rbac.spec.ts	CT009	Paciente não acessa /profissionais (redirecionado)	Paciente
rbac.spec.ts	CT010	Profissional não acessa /pacientes (redirecionado)	Profissional
rbac.spec.ts	CT011	Usuário não logado é redirecionado para /login	Anônimo
lgpd.spec.ts	CT012	Banner LGPD aparece e desaparece após aceitar	Anônimo
relatorios.spec.ts	CT013	Admin acessa página de relatórios com métricas	Admin
telemedicina.spec.ts	CT014	Paciente acessa telemedicina e vê seletor de consulta	Paciente
prontuario-registro.spec.ts	CT015	Profissional adiciona registro clínico ao prontuário	Profissional
receita-emissao.spec.ts	CT016	Profissional emite receita digital para paciente	Profissional
receita-paciente.spec.ts	CT017	Paciente visualiza suas receitas prescritas	Paciente
profissional-crud.spec.ts	CT018	Admin cadastra novo profissional de saúde	Admin
paciente-edicao.spec.ts	CT019	Admin edita dados de um paciente existente	Admin

Arquivo	ID	Descrição	Perfil
consulta-conclusao.spec.ts	CT020	Profissional conclui uma consulta agendada	Profissional

Execução

```
npm run dev# Terminal 1: iniciar o servidor npm run test:e2e# Terminal 2: executar testes
```

Resultado

Banco resetado com seed (pré-testes)

```
login-admin.spec.ts > login do admin redireciona para o dashboard
login-admin.spec.ts > admin vê links de pacientes, profissionais e relatórios
login-profissional.spec.ts > login do profissional redireciona para o dashboard
login-profissional.spec.ts > profissional vê links de agendamentos...
login-paciente.spec.ts > login do paciente redireciona para o dashboard
cadastro-paciente.spec.ts > admin cadastra paciente com CPF válido
agendamento.spec.ts > paciente agenda consulta
rbac.spec.ts > paciente não acessa /pacientes (redirecionado)
rbac.spec.ts > paciente não acessa /profissionais (redirecionado)
rbac.spec.ts > profissional não acessa /pacientes (redirecionado)
rbac.spec.ts > usuário não logado é redirecionado para /login
lgpd.spec.ts > banner LGPD aparece na primeira visita e desaparece após aceitar
relatorios.spec.ts > admin acessa página de relatórios com métricas
telemedicina.spec.ts > paciente acessa telemedicina e vê seletor de consulta
prontuario-registro.spec.ts > profissional adiciona registro clínico
receita-emissao.spec.ts > profissional emite receita digital
receita-paciente.spec.ts > paciente visualiza suas receitas prescritas
profissional-crud.spec.ts > admin cadastra novo profissional
paciente-edicao.spec.ts > admin edita dados de um paciente existente
consulta-conclusao.spec.ts > profissional conclui uma consulta agendada
```

Banco restaurado com seed (pós-testes)

20 passed

Cobertura por requisito

Requisito	Testes que cobrem
RF001 – Cadastro de pacientes	CT006
RF002 – Agendamentos	CT007
RF003 – Prontuário	CT004, CT015
RF004 – Teleconsulta	CT014

Requisito	Testes que cobrem
RF005 – RBAC	CT001-CT005, CT008-CT011
RF006 – Receitas	CT004, CT016, CT017
RF007 – Profissionais (CRUD)	CT018
RF011 – Edição de pacientes	CT019
RNF002 – LGPD	CT012
RNF006 – Validação Zod	32 testes unitários

6. Conclusão

Resultados alcançados

O **VidaPlus** cumpre o objetivo de demonstrar um MVP funcional do Sistema de Gestão Hospitalar e de Serviços de Saúde (SGHSS). Todos os **11 requisitos funcionais** e **7 requisitos não funcionais** identificados foram implementados com sucesso.

A aplicação oferece fluxos completos para os três perfis de usuário – administrador, paciente e profissional de saúde – incluindo cadastro, agendamento, prontuário eletrônico, receitas digitais e **teleconsulta real via Jitsi Meet**.

A cobertura de testes totaliza **32 testes unitários** (Vitest) e **20 testes E2E** (Playwright), validando schemas de dados, fluxos críticos, controle de acesso, CRUD de profissionais, edição de pacientes, emissão e visualização de receitas, prontuário e conformidade LGPD.

Lições aprendidas

- **Server Actions** do Next.js simplificam formulários e mutações de dados, eliminando a necessidade de uma API REST separada para o escopo do MVP
- **Auth.js com JWT** atende o controle de perfis de forma eficaz sem a complexidade de OAuth
- **Prisma com SQLite** oferece produtividade no desenvolvimento e facilidade de setup, ideal para protótipos
- **Jitsi Meet** é uma alternativa viável e gratuita para videoconferência, embora o servidor público (`meet.jit.si`) imponha limitações como autenticação de moderador
- **Testes E2E com Playwright** validam fluxos críticos rapidamente, mas exigem cuidado com seletores e condições de corrida (race conditions)
- **Zod** permite compartilhar validações entre cliente e servidor, garantindo consistência
- O padrão de **seed + teardown** para testes E2E evita poluição de dados entre ambiente de desenvolvimento e testes

Evoluções futuras

Evolução	Justificativa
Banco PostgreSQL (Neon/Turso)	Persistência confiável para produção
OAuth (Google/GitHub)	Login social para conveniência do usuário
Auditoria LGPD completa	Registro de consentimento, exportação e exclusão de dados
Jitsi self-hosted ou Daily.co	Eliminar limitação de autenticação do servidor público
Upload de exames (S3/R2)	Anexar PDFs e imagens aos prontuários
Notificações por e-mail	Lembrete de consultas via SMTP

Evolução	Justificativa
NestJS separado	Backend dedicado caso a aplicação escale para microserviços
CI/CD com GitHub Actions	Build, lint, testes automatizados em cada PR

Referências

- NEXT.JS. **Documentação oficial**. Disponível em: <https://nextjs.org/docs>. Acesso em: maio 2026.
- PRISMA. **Documentação oficial**. Disponível em: <https://www.prisma.io/docs>. Acesso em: maio 2026.
- AUTH.JS. **Documentação oficial (v5)**. Disponível em: <https://authjs.dev>. Acesso em: maio 2026.
- JITS MEET. **Handbook**. Disponível em: <https://jitsi.github.io/handbook>. Acesso em: maio 2026.
- PLAYWRIGHT. **Documentação oficial**. Disponível em: <https://playwright.dev/docs>. Acesso em: maio 2026.
- VITEST. **Documentação oficial**. Disponível em: <https://vitest.dev>. Acesso em: maio 2026.
- SHADCN/UI. **Componentes**. Disponível em: <https://ui.shadcn.com>. Acesso em: maio 2026.
- BRASIL. **Lei nº 13.709/2018 — Lei Geral de Proteção de Dados Pessoais (LGPD)**. Disponível em: https://www.planalto.gov.br/ccivil_03/_ato2015-2018/2018/lei/l13709.htm.
- UNINTER. **Roteiro do Projeto Multidisciplinar — SGHSS**. Centro Universitário Internacional, 2025.